

University of Verona
A.Y. 2025-26

Machine Learning & Deep Learning

Deep Learning

Beyond Supervised Learning

Semi-Supervised Learning, Pseudo-Labelling and Noisy Labels

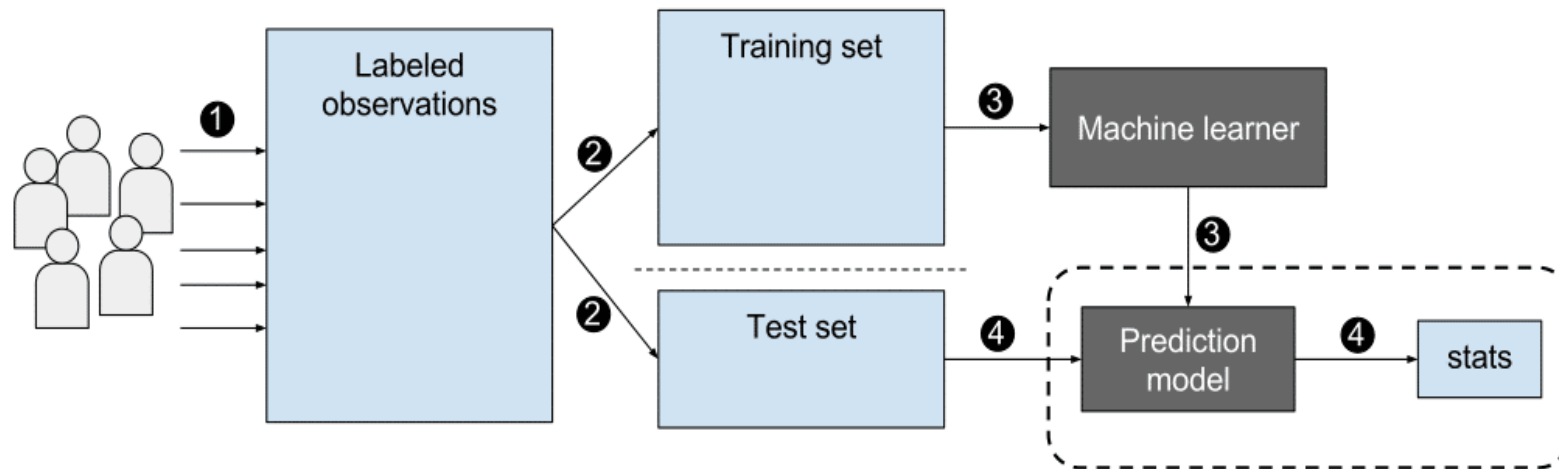
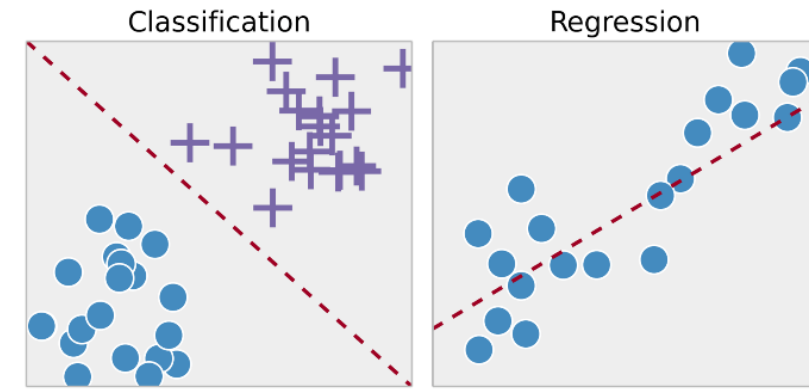
Vittorio Murino

Credits

- Waqar Ahmed
- <https://towardsdatascience.com/pseudo-labeling-to-deal-with-small-datasets-what-why-how-fd6f903213af>

Supervised Learning

- A supervised learning algorithm learns from labeled training data, helps you to predict outcomes for unforeseen data.
- It primarily covers two kinds of tasks:
 - **Classification:** asks the algorithm to predict a discrete value
 - **Regression:** approximates a mapping function (f) from input variables (X) to a continuous output variable (y).



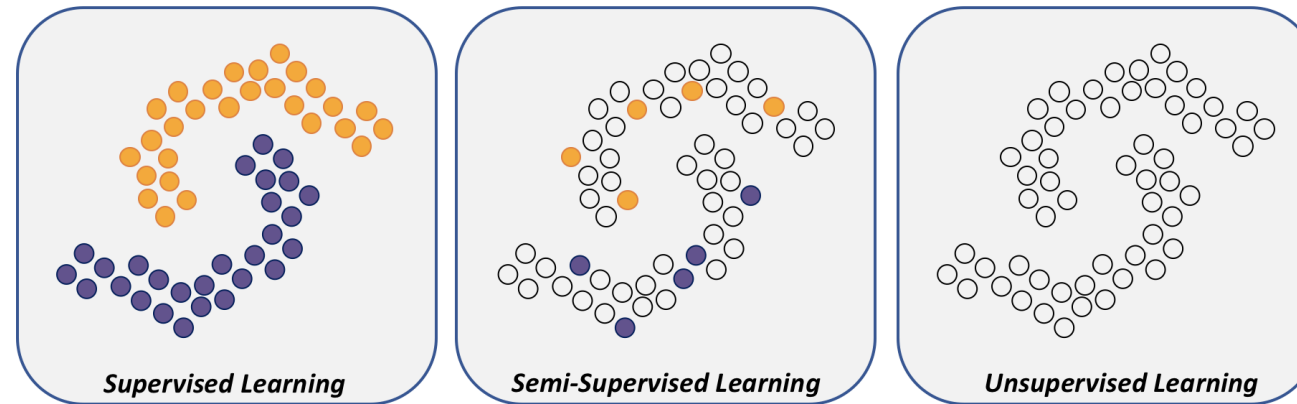
Unsupervised Learning

- The training dataset contains examples without a specific desired outcome or a label.
- A Machine Learning model attempts to automatically find structure in the data by extracting useful features and analyzing them.
- It primarily covers following tasks:
 - **Clustering:** deals with finding a structure or pattern in a collection.
 - **Associations:** discovers exciting relationships between variables in large databases. For example, people that buy a new home most likely to buy new furniture.



Semi-Supervised Learning

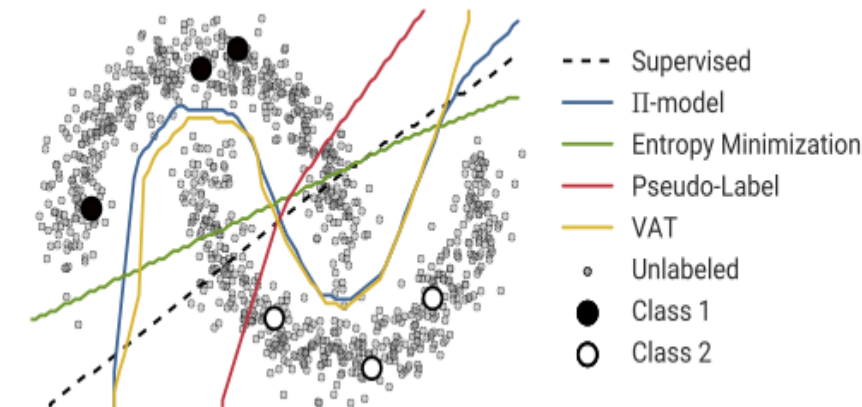
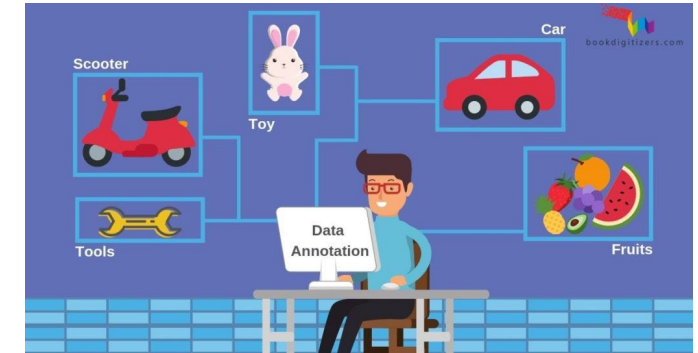
- The approach that deals in between the two extremes (supervised where entire dataset is labeled and unsupervised where there are no labels).



- Typically, SSL is performed using a small labeled dataset and a relatively larger unlabeled dataset.
- *The goal is to learn a predictor that predicts future test data better than the predictor learned from the labeled training data alone.*

Why should we care about Semi-Supervised Learning?

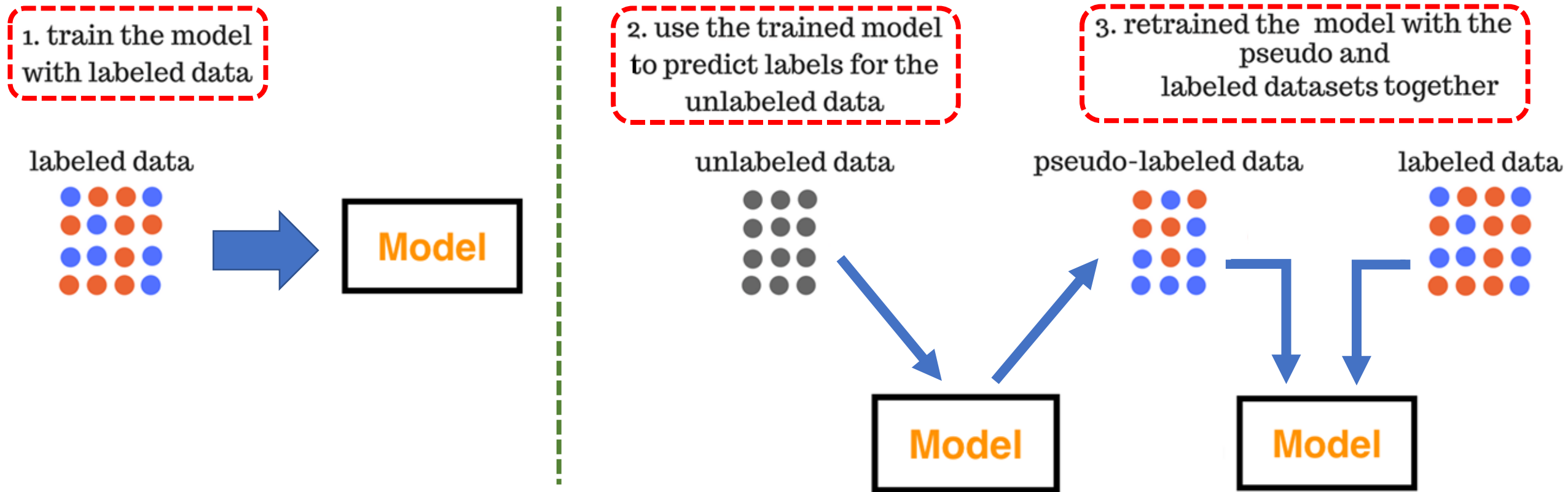
- In many real-world applications, it is either too expensive or not feasible to collect large labeled datasets.
- But a large volume of unlabeled data can be available.
- For such scenarios, SSL is a perfect fit as it can leverage the labeled data and also derive structure from the unlabeled data to solve the overall task better.
- Let's take an example of model when trained on only the labeled data, the decision boundary (dashed line) does not follow the contours of the data, as indicated by additional unlabeled data (small grey dots).
- So, the objective of SSL is to utilize the unlabeled data to produce a decision boundary that better reflects the data's underlying structure.



Semi-Supervised Learning and Pseudo Labeling

Pseudo-Labeling: A Naive Semi-Supervised Learning Method

First proposed by [Lee in 2013](#), in which network is trained in a supervised fashion with labeled and unlabeled data simultaneously.



Cont'd

- For unlabeled data, just picking up the class which has the maximum predicted probability – *pseudo-labels* – and use that as if they were true labels.

$$y'_i = \begin{cases} 1 & \text{if } i = \operatorname{argmax}_{i'} f_{i'}(x) \\ 0 & \text{otherwise} \end{cases}$$

- Because the total number of labeled data and unlabeled data is quite different and the training balance between them is quite important for the network performance, the overall loss function is

$$L = \frac{1}{n} \sum_{m=1}^n \sum_{i=1}^C L(y_i^m, f_i^m) + \alpha(t) \frac{1}{n'} \sum_{m=1}^{n'} \sum_{i=1}^C L(y_i'^m, f_i'^m),$$

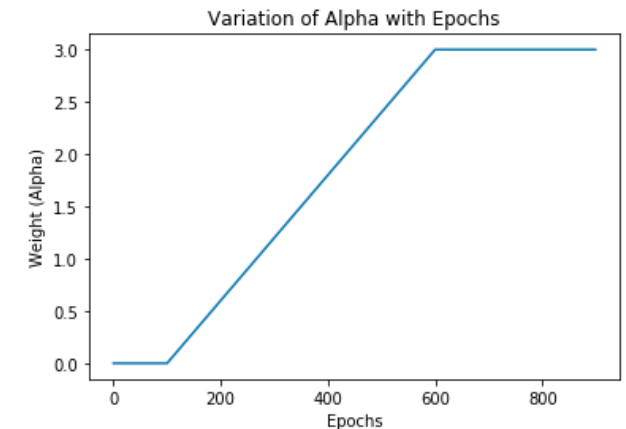
- Or in simpler words:

$$\text{Loss per Batch} = \text{Labeled Loss} + \text{Weight} * \text{Unlabeled Loss}$$

Cont'd

- In the equation, the weight α is used to control the contribution of unlabelled data to the overall loss.
- Moreover, the weight is a function of time (epochs) and is slowly increased during training.
- Lee proposes using the following equation for alpha as a function of t :

$$\alpha(t) = \begin{cases} 0 & t < T_1 \\ \frac{t-T_1}{T_2-T_1} \alpha_f & T_1 \leq t < T_2 \\ \alpha_f & T_2 \leq t \end{cases}$$

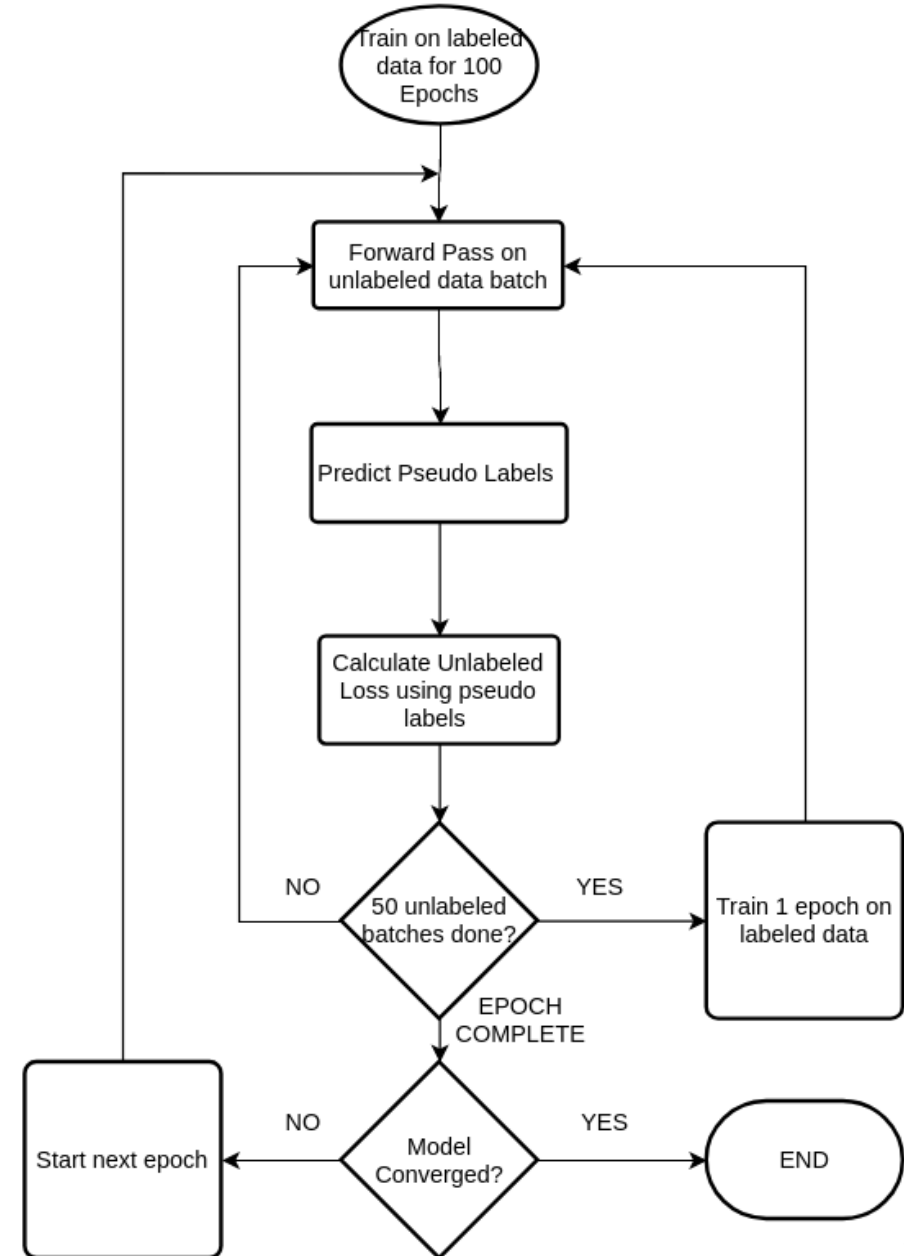


- In the first T_1 epochs (100 in this case) the weight is 0, effectively forcing the model to train only on the labelled data.
- After T_1 epochs, the weight linearly increases to α_f (3 in this case) until T_2 epochs (600 in this case), this allows the model to slowly incorporate the unlabelled data.
- T_2 and α_f control the rate at which the weight increases and the value after saturation respectively.

Pseudo-Labeling implementation

This alternation between labelled and unlabelled data training helps in 2 ways:

1. It reduces overfitting on the labeled training data
2. Improves speed since we need to make only 1 forward pass per batch (on the unlabeled data) instead of 2 (unlabeled and labeled) as mentioned in the paper.



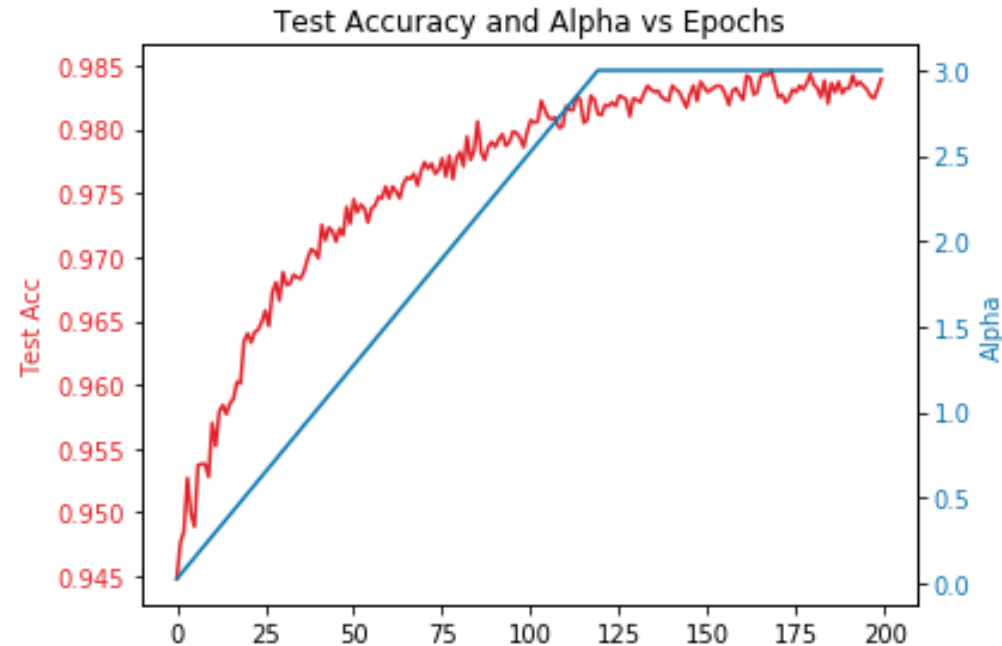
Baseline performance

- MNIST dataset: digits, 10 classes
- We use 1000 labeled images (class balanced) and 59,000 unlabeled images for the training set and 10,000 images for the test set.
- The performance on the 1000 labeled images without using any of the unlabeled images (i.e. simple supervised training) is
 - Epoch: 290 | Train Loss : 0.00004 | Test Acc : 95.57 | Test Loss : 0.233
- With 1000 labeled images the best test accuracy is 95.57%, which is the baseline

PL performance

- Here's the result after training 100 epochs on labeled data followed by 170 epochs of semi-supervised training:
- # Best Accuracy is at 168 epochs:
Epoch: 168 | Alpha Weight : 3.00000 | Test Acc : 98.46000 | Test Loss : 0.075
- After using the unlabelled data we reached an accuracy of 98.46% that's ~ 3% more than with supervised training.
- But how PL is actually working? Let's see some visualizations ...

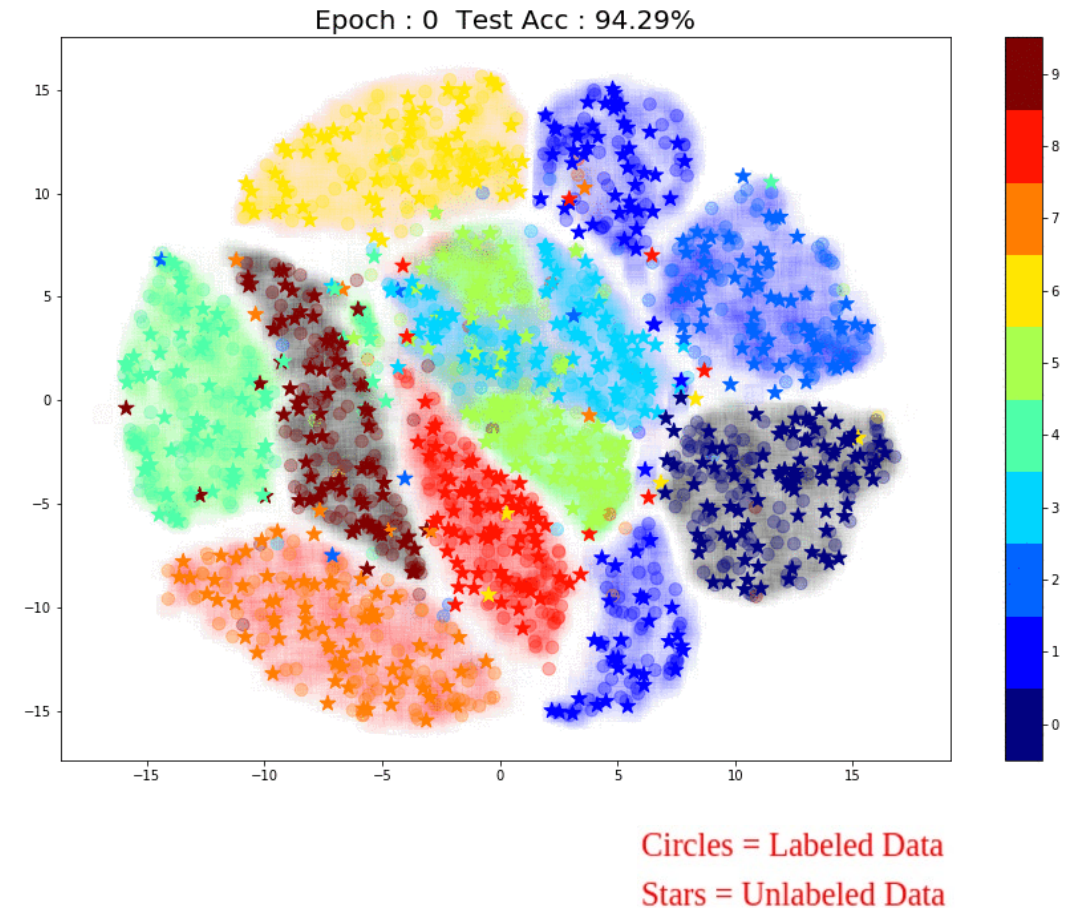
Alpha Weight vs. Accuracy



It's clear that as alpha increases the test accuracy also slowly increases and later saturates

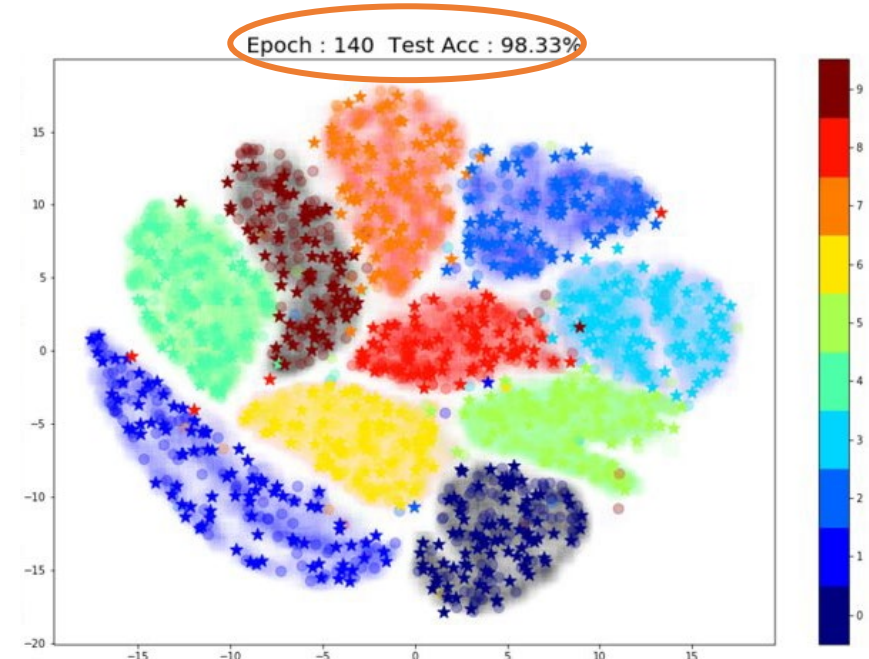
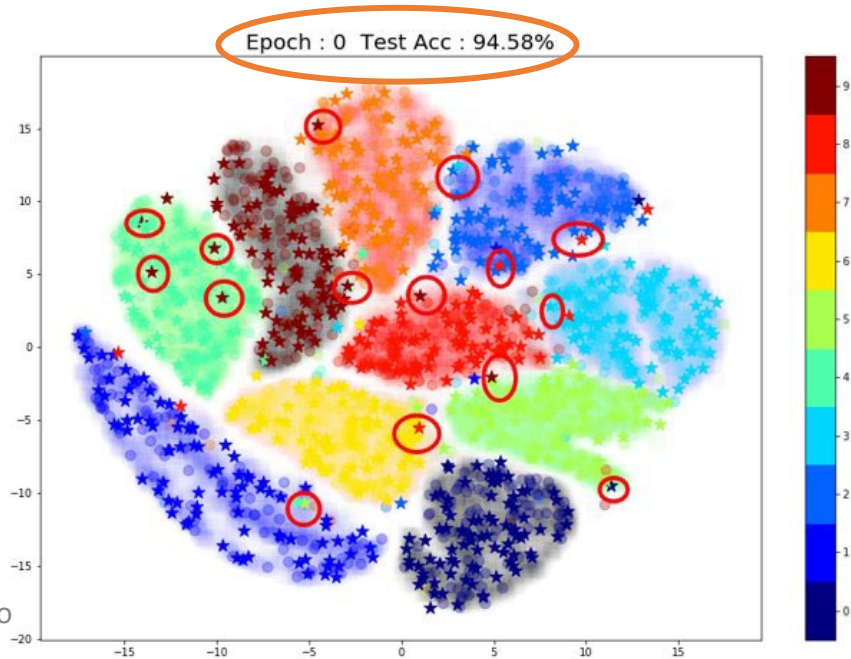
T-SNE Visualization

- Now let's have a look at how the pseudo labels are being assigned at every epoch. In the plot, there are 3 things to note:
 - The faint colour in the background of each cluster is the true label. This is created using T-SNE of all 60k training images (labels used)
 - The small circles inside each cluster are from the 1000 training images that were used in the supervised training phase.
 - The small stars that keep moving are the pseudo labels that the model assigns for the unlabelled images for each epoch. (For each epoch, around 750 randomly sampled unlabelled images are used to create the plot)



T-SNE Visualization

- Here are some things to notice:
 - Most of the pseudo-labels are correct. (Stars are in clusters with the same colour) This can be attributed to the high initial test accuracy.
 - As training continues, the percentage of correct pseudo labels increases. This is reflected in the increased overall test accuracy of the model.
 - Here's a plot that shows the **same** 750 points at Epoch 0 (left) and Epoch 140 (right).
 - The points that have improved are marked in red circles.



Why does Pseudo-Labeling work?

- The goal of any Semi-Supervised Learning algorithm is to use both the unlabelled and labelled samples to learn the underlying structure of the data. Pseudo-Labeling is able to do this by making two important assumptions:
 - **Continuity Assumption (Smoothness):** *Points that are close to each other are more likely to share a label.* In other words, small changes in input **do not** cause large changes in output. This assumption allows pseudo labelling to conclude that small changes in images like rotation, shearing, etc do not change the label.
 - **Cluster Assumption:** *The data tend to form discrete clusters, and points in the **same cluster** are more likely to **share a label**. This is a special case of the **continuity assumption**. Another way to look at this is the decision boundary between classes lies in the low-density region (*doing so helps in generalization — similar to maximum margin classifiers in SVM*).*

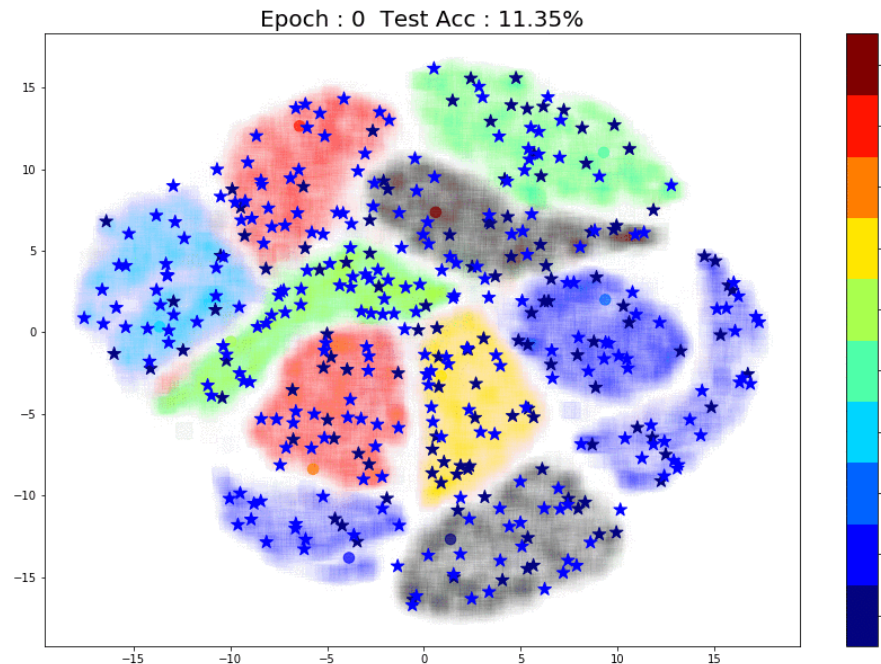
Why does Pseudo-Labelling work?

- Therefore, the initial labelled data is important — it helps the model learn the underlying cluster structure.
- When we assign a pseudo label, we are using the cluster structure that the model has learned to infer labels for the unlabelled data. As the training progresses, the learned cluster structure is improved using the unlabelled data.
- If the initial labelled data is too small in size or contains outliers, pseudo labelling will likely assign incorrect labels to the unlabelled points.
- The opposite also holds, i.e., pseudo labelling can benefit from a classifier that is already performing well with just the labelled data.
- This should make more sense when we look at scenarios where pseudo-labelling fails.

When does Pseudo-Labelling not work well?

Case 1

- Initial Labelled data is not enough to determine clusters
- To understand this scenario better, instead of using 1000 initial points let's take the extreme case and use just 10 labelled points and see how pseudo-labelling performs

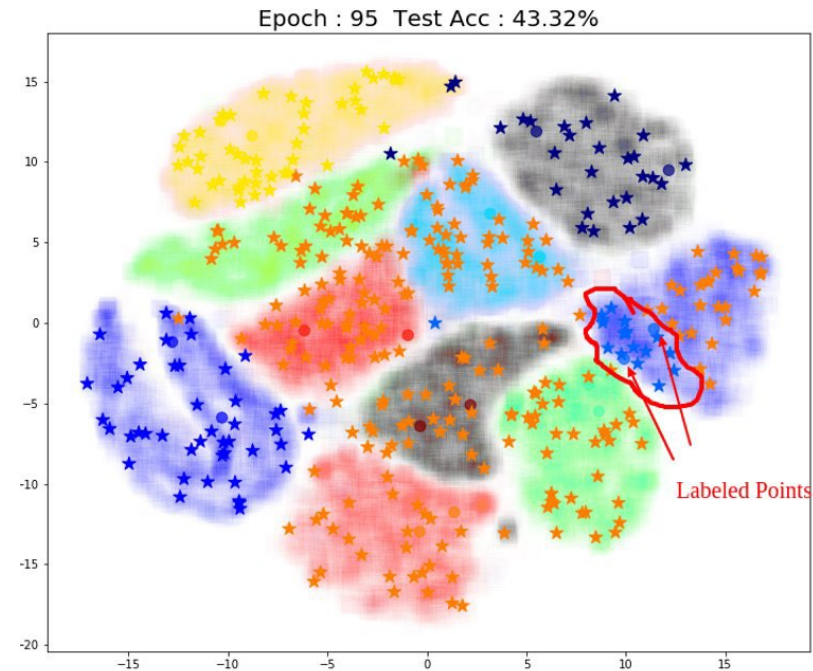
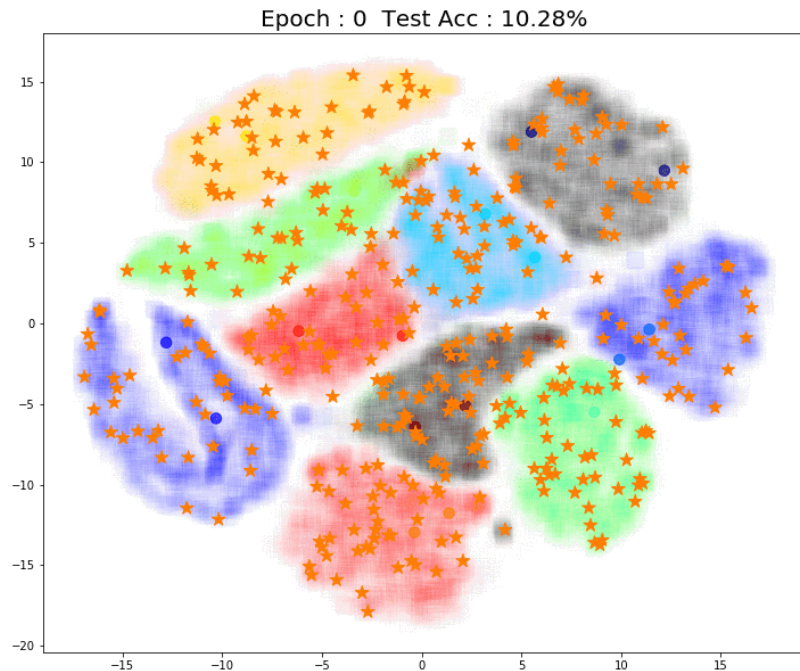


- As expected, pseudo-labelling has almost no difference. The model itself is as good as a random model with 10% accuracy. Since each class has just 1 point, the model is incapable of learning the underlying structure for any class

When does Pseudo-Labeling not work well?

Case 1

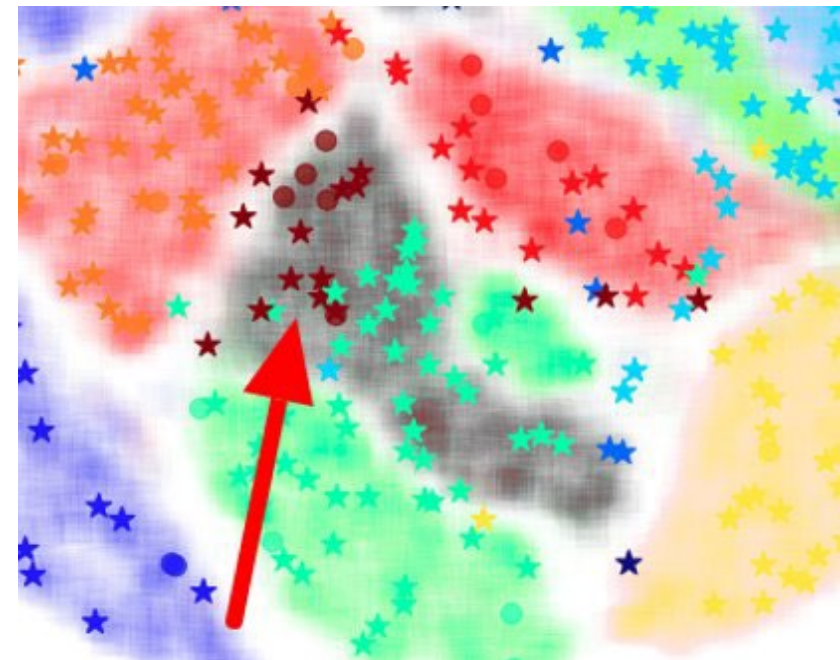
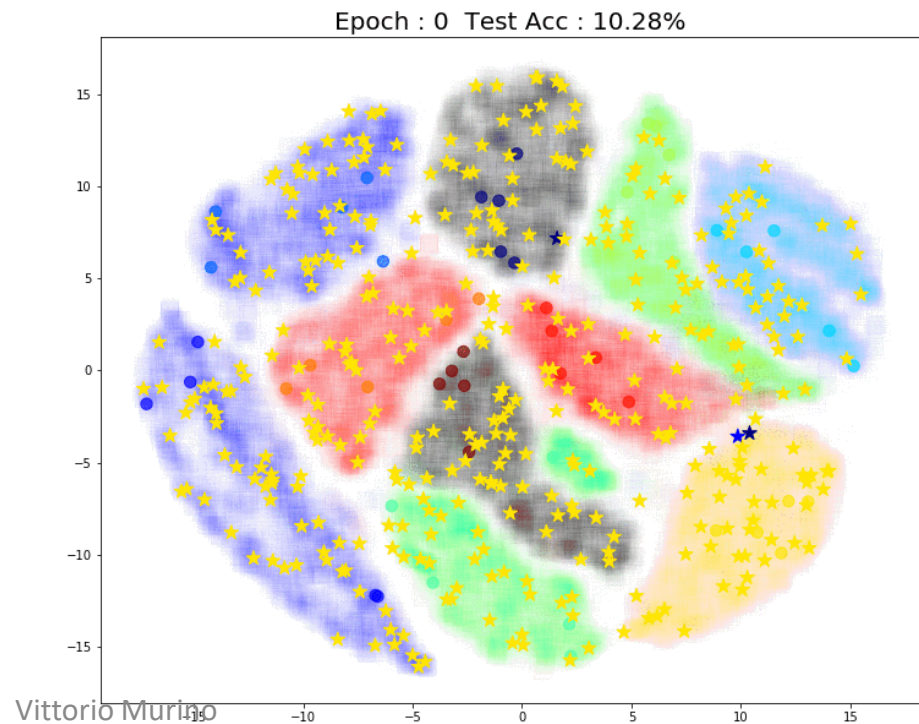
- Let's increase the number of labelled points to 20 (2 points per class, figure on the left)
- Now the model is performing slightly better as it learns the structure for some classes. Here's something interesting — notice that pseudo-labelling assigns the correct labels for these points (circled in red in the figure on the right) most likely because there are two labelled points close-by.



When does Pseudo-Labeling not work well?

Case 1

- And finally, let's try 50 points: (5 points per class, figure on left)
- The performance is much better! And once again, notice the small group of brown labelled points right in the centre of the images. The points in the same brown cluster but further away from the labelled points are always incorrectly predicted as Aqua green ('4') or Orange ('7').



When does Pseudo-Labelling not work well?

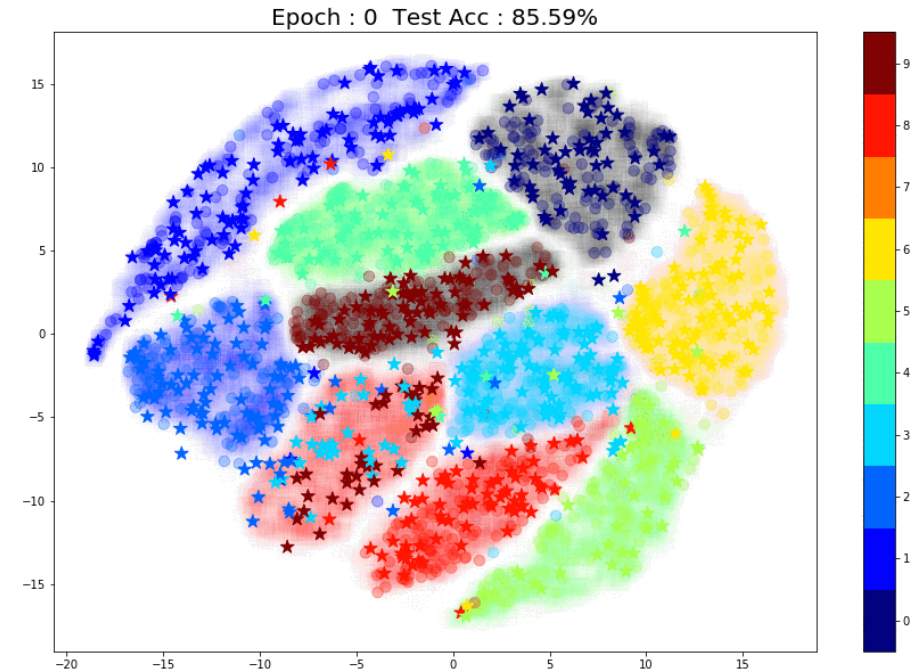
Case 1

- A few things to note:
 - For all the above experiments (10, 20 and 50 points) the way the labelled points were chosen made a huge difference. Any outliers completely changed the model's performance and predictions for pseudo-labels. This is a common problem with small datasets.
 - While T-SNE is a great tool for visualization, we need to keep in mind that it is *probabilistic* and merely gives us *an idea* of how clusters might be distributed in higher-dimensional space.
 - To conclude, both the quantity and quality of initial labelled points make a difference when it comes to pseudo-labelling. Further, the model might require different amounts of data for different classes to understand that particular class's structure.

When does Pseudo-Labeling not work well?

Case 2

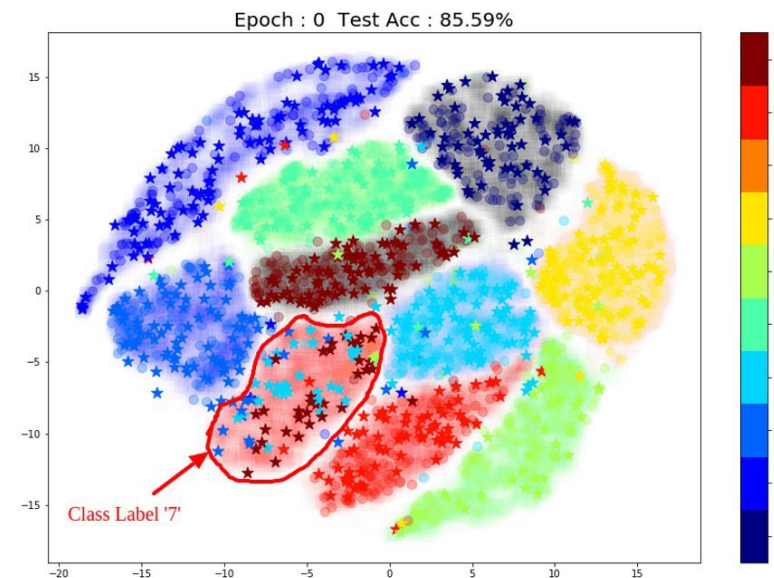
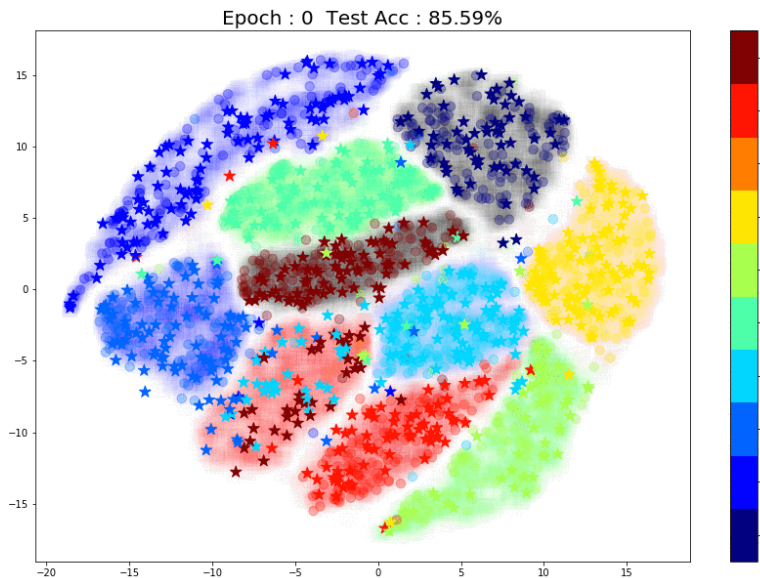
- The case in which initial labelled data does not include some classes.
- Let's see what happens if the labelled dataset does not contain one class (e.g.: '7' not included in the labelled set, but the unlabelled data still retains all classes)
- After training 100 epochs on the labelled data:
Test Acc : 85.63000 | Test Loss : 1.555
- And after semi-supervised training :
Epoch: 99 | Alpha Weight : 2.5 | Test Acc : 87.98 | Test Loss : 2.98



When does Pseudo-Labeling not work well?

Case 2

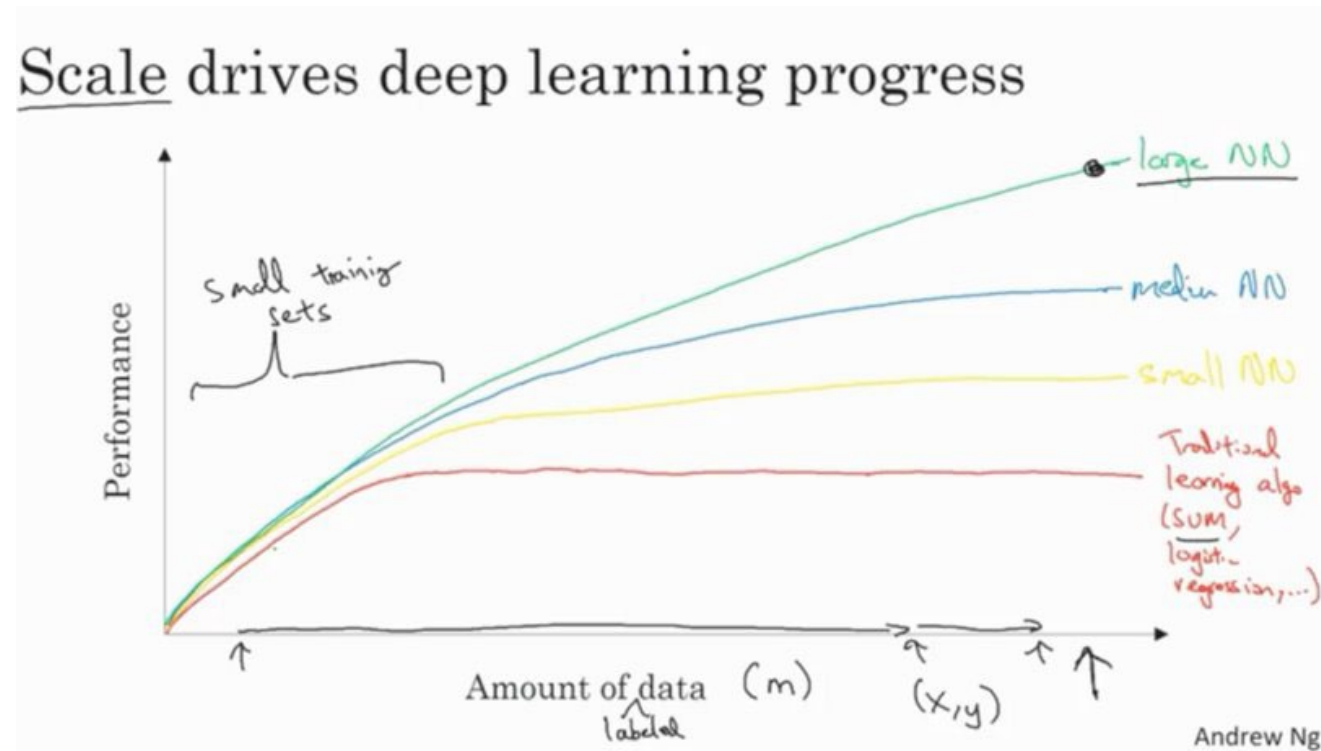
- The overall accuracy does increase from 85.6% to 87.98% but does not show any improvements after that. This is obviously because the model is unable to learn the cluster structure for the class label '7'.
- It's no surprise that pseudo-labelling struggles here as our model does not have the capability to learn about classes that it has never seen before.
- However, over the past few years, a lot of interest has been shown in Zero-Shot Learning techniques which enable models to recognize labels even if they do not exist in the training data.



When does Pseudo-Labeling not work well?

Case 3

- No benefit from increased data
- In some cases, the model might not have enough complexity to take advantage of the additional data. This usually happens when using pseudo-labelling with conventional ML algorithms like Logistic Regression or SVMs. When it comes to Deep Learning models, large DL models almost always benefit from having more data



Challenges with Semi-Supervised Learning

Combining Unlabelled Data with Labelled Data

- The primary objective of Semi-Supervised Learning is to use the unlabelled data along with the labelled data to understand the underlying structure of the dataset.
The obvious question here is: How to utilize the unlabelled data to achieve this purpose?
- In the Pseudo-Labeling technique, we saw that a scheduled weight function (α) was used to slowly combine the unlabelled data with the labelled data. However, the function $\alpha(t)$ assumes that the model confidence increases over time and therefore increases the unlabelled loss linearly.
- This needs not be the case as model predictions can sometimes be incorrect. In fact, if the model makes several wrong unlabelled predictions, pseudo-labelling can act like a bad feedback loop and deteriorate performance further (Arazo et al., 2019).
- One solution for the problem above is to use probability thresholds, similar to what it's done with Logistic Regression.
- Other Semi-Supervised Learning algorithms use different ways to combine the data, for example, MixMatch uses a 2-step process for guessing the label for the unlabeled data, followed by Mixup data augmentation to combine the unlabeled data with the labeled data (Berthelot et al., 2019).

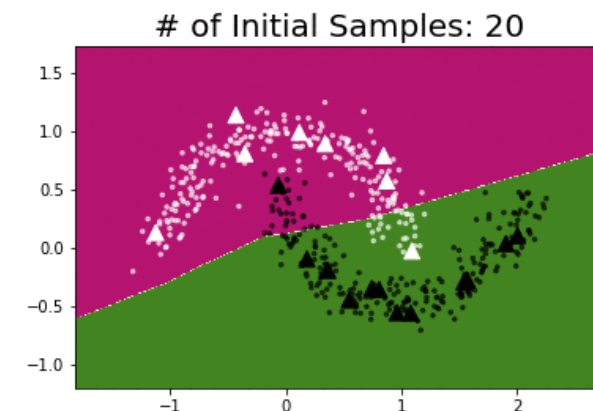
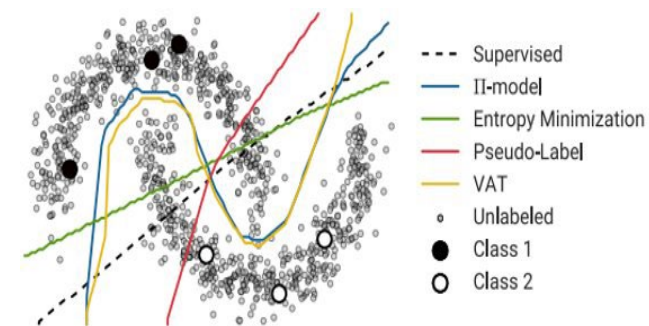
E. Arazo et al. "Pseudo-labeling and confirmation bias in deep semi-supervised learning." *Int'l Joint Conference on Neural Networks IJCNN 2020*.

D. Berthelot et al. "Mixmatch: A holistic approach to semi-supervised learning." *NeurIPS 2019*.

Challenges with Semi-Supervised Learning

Data Efficiency

- Another challenge with Semi-Supervised Learning is to design algorithms that can work with very small amounts of labelled data. As we have seen with pseudo labelling, the model works best with 1000 initial labelled samples. However, when the labelled dataset is reduced further (e.g., 50 points), pseudo-labelling's performance starts to drop.
- Oliver et al. (2018) did a comparison of several Semi-Supervised Learning algorithms and found that Pseudo-Labeling fails on the “two-moons” dataset while other models like VAT and Pi-Model worked much better.
- As shown in the image, VAT (Miyato et al., 2019) and Pi-Model (Samuli & Aila, 2017) learn a decision boundary that is surprisingly good with just 6 labelled data points (shown in large white and black circles). Pseudo-Labeling on the other hand completely fails and learns a linear decision boundary instead.
- The experiment was repeated using the same model of *Oliver et al.*, and found that pseudo-labelling required anywhere from 30–50 labelled points (depending on the position of the labelled points) to learn the underlying data structure.
- To make Semi-Supervised Learning more practical we need algorithms that are highly data-efficient i.e., ones that can work on very small amounts of labelled data.



A. Oliver et al. "Realistic evaluation of deep semi-supervised learning algorithms." *NeurIPS* 2018.

T. Miyato et al. "Virtual adversarial training: a regularization method for supervised and semi-supervised learning." *TPAMI* 2019.

L. Samuli and T. Aila. "Temporal ensembling for semi-supervised learning." *ICLR* 2017.

Conclusions

- In summary, *pseudo-labeling is a simple heuristic which is widely used in practice, likely because of its simplicity and generality*, and as we have seen it provides a nice way to learn about Semi-Supervised Learning.
- Over the last 2–3 years, Semi-Supervised Learning for image classification has seen some incredible improvements.
 - Unsupervised Data Augmentation (Xie et al., 2020) has achieved 97.3% on CIFAR-10 with just 4000 Labels.
 - To put that into perspective, DenseNet (Huang et al., 2017) achieved 96.54% on the complete CIFAR-10 dataset in 2017.
- It's really interesting to see how the Machine Learning and Data Science community is moving towards algorithms that either uses less labelled data (like Semi-Supervised Learning, Zero/Few-Shot Learning) or smaller datasets altogether (like Transfer Learning).

A. Oliver et al. "Realistic evaluation of deep semi-supervised learning algorithms." *NeurIPS* 2018.

Qizhe Xie et al. "Unsupervised data augmentation for consistency training." *NeurIPS* 2020.

Gao Huang et al. "Densely connected convolutional networks." *IEEE CVPR* 2017.

Ensembling methods, mutual/co-teaching, and noisy labels

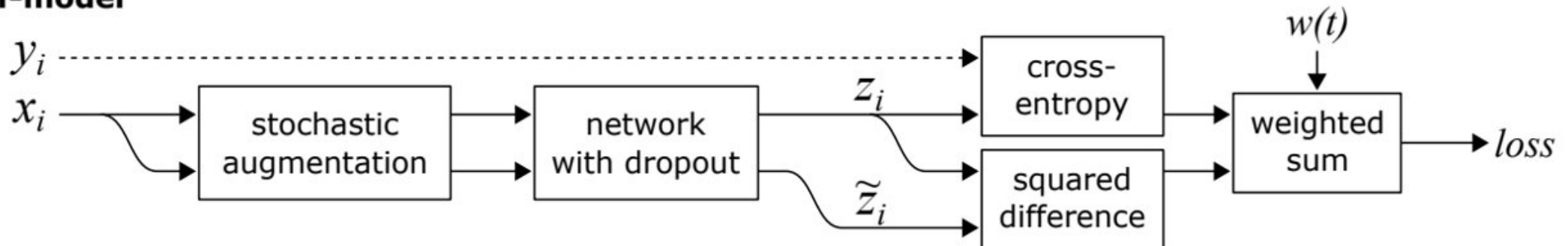
Ensembling for Semi-Supervised Learning

- A simple and efficient method for training deep neural networks in a semi-supervised setting where only a small portion of training data is labelled.
- Authors introduce self-ensembling, where they form a **consensus prediction of the unknown labels** using the **outputs of the network-in-training on different epochs**.
- The training still operates on a single network, but the predictions made on different epochs correspond to an ensemble prediction of a large number of individual sub-networks because of dropout regularization.
- Such ensemble prediction can be expected to be a better predictor for the unknown labels than the output of the network at the most recent training epoch.
- Therefore, the labels inferred this way can be used as training targets for the unlabeled inputs.
- The method relies heavily on dropout regularization and versatile input augmentation.
- We describe two ways to implement self-ensembling, **Π -model** and **temporal ensembling**. Both approaches surpass prior state-of-the-art results in semi-supervised learning by a considerable margin.

Π -model

- Encourages consistent network output between two realizations of the same input stimulus, under two different dropout conditions.
- During training, we evaluate the network for each training input x_i twice, resulting in prediction vectors z_i and $\tilde{z}_i$.
- The loss function consists of two components.
 - The first component is the standard cross-entropy loss, evaluated for labeled inputs only.
 - The second component, evaluated for all inputs, penalizes different predictions for the same training input x_i by taking the mean square difference between the prediction vectors z_i and $\tilde{z}_i$.

Π -model



Π -model

- To combine the supervised and unsupervised loss terms, we scale the latter by time-dependent weighting function $w(t)$.
- By comparing the entire output vectors z_i and $\tilde{z}_i$, we effectively ask the “dark knowledge” between the two evaluations to be close, which is a much stronger requirement compared to asking that only the final classification remains the same, which is what happens in traditional training.

Algorithm 1 Π -model pseudocode.

Require: x_i = training stimuli

Require: L = set of training input indices with known labels

Require: y_i = labels for labeled inputs $i \in L$

Require: $w(t)$ = unsupervised weight ramp-up function

Require: $f_\theta(x)$ = stochastic neural network with trainable parameters θ

Require: $g(x)$ = stochastic input augmentation function

for t in $[1, num_epochs]$ **do**

for each minibatch B **do**

$z_{i \in B} \leftarrow f_\theta(g(x_{i \in B}))$

$\tilde{z}_{i \in B} \leftarrow f_\theta(g(x_{i \in B}))$

$loss \leftarrow -\frac{1}{|B|} \sum_{i \in (B \cap L)} \log z_i[y_i]$
 $+ w(t) \frac{1}{C|B|} \sum_{i \in B} \|z_i - \tilde{z}_i\|^2$

 update θ using, e.g., ADAM

end for

end for

return θ

▷ evaluate network outputs for augmented inputs

▷ again, with different dropout and augmentation

▷ supervised loss component

▷ unsupervised loss component

▷ update network parameters

Π -model

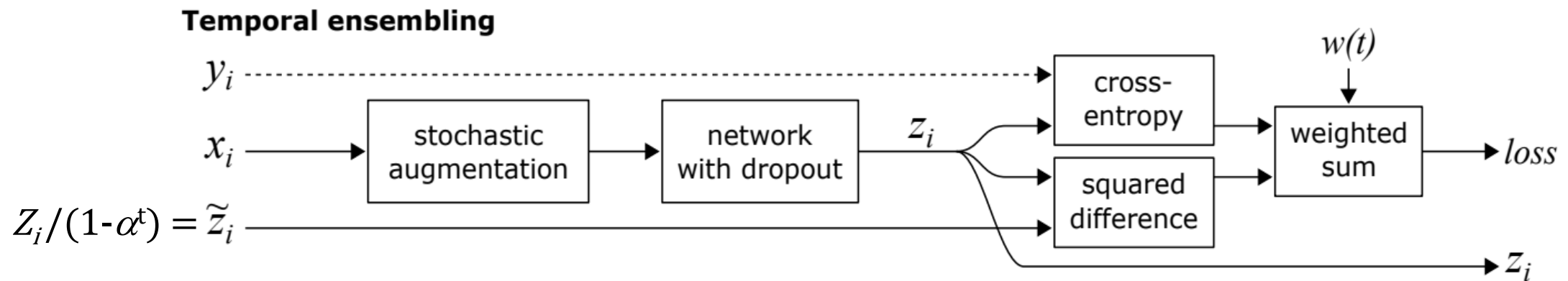
- It is important to note that, because of dropout regularization, the network output during training is stochastic. Thus, two evaluations of the same input x_i under same network weights θ may yield different results.
- In addition, Gaussian noise and augmentations such as random translation are evaluated twice, resulting in additional variations. The combination of these effects explains the difference between the prediction vectors z_i and $\tilde{z}_i$.
- This difference can be seen as an error in classification, given that the original input x_i was the same, and thus minimizing it is a reasonable goal.
- In the implementation, the unsupervised loss weighting function $w(t)$ ramps up, starting from zero, along a Gaussian curve during the first 80 training epochs.
- In the beginning the total loss and the learning gradients are thus dominated by the supervised loss component, i.e., the labeled data only.
- We have found it to be very important that the ramp-up of the unsupervised loss component is slow enough — otherwise, the network gets easily stuck in a degenerate solution where no meaningful classification of the data is obtained.

Temporal Ensembling

- Analyzing how the Π -model works, we could equally well split the evaluation of the two branches in two separate phases:
 - first classifying the training set once without updating the weights θ , and
 - second, training the network on the same inputs under different augmentations and dropout, using the just obtained predictions as targets for the unsupervised loss component.
- As the training targets obtained this way are based on a single evaluation of the network, they can be expected to be noisy.
- Temporal ensembling alleviates this by aggregating the predictions of multiple previous network evaluations into an ensemble prediction.
- It also let us evaluate the network only once during training, gaining an approximate 2x speedup over the Π -model.

Temporal Ensembling

- After every training epoch, the network outputs z_i are accumulated into ensemble outputs Z_i by
 - updating $Z_i \leftarrow \alpha Z_i + (1 - \alpha) z_i$, where α is a momentum term that controls how far the ensemble reaches into training history.
- Because of dropout regularization and stochastic augmentation, Z thus contains a weighted average of the outputs of an ensemble of networks f from previous training epochs, with recent epochs having larger weight than distant epochs.
- For generating the training targets $\tilde{z}_i$, we need to correct for the startup bias in Z by dividing by factor $(1 - \alpha^t)$.



Temporal Ensembling

- On the first training epoch, Z and $\tilde{z}_i$ are zero as no data from previous epochs is available.
- For this reason, we specify the unsupervised weight ramp-up function $w(t)$ to also be zero on the first training epoch.
- In all runs, we ramped up learning rate λ and unsupervised loss component weight w during the first 80 epochs using a Gaussian ramp-up curve $\exp[-5(1 - T)^2]$, where T advances linearly from zero to one during the ramp-up period.

Algorithm 2 Temporal ensembling pseudocode. Note that the updates of Z and $\tilde{z}$ could equally well be done inside the minibatch loop; in this pseudocode they occur between epochs for clarity.

Require: x_i = training stimuli

Require: L = set of training input indices with known labels

Require: y_i = labels for labeled inputs $i \in L$

Require: α = ensembling momentum, $0 \leq \alpha < 1$

Require: $w(t)$ = unsupervised weight ramp-up function

Require: $f_\theta(x)$ = stochastic neural network with trainable parameters θ

Require: $g(x)$ = stochastic input augmentation function

$Z \leftarrow \mathbf{0}_{[N \times C]}$ ▷ initialize ensemble predictions

$\tilde{z} \leftarrow \mathbf{0}_{[N \times C]}$ ▷ initialize target vectors

for t in $[1, \text{num_epochs}]$ **do**

for each minibatch B **do**

$z_{i \in B} \leftarrow f_\theta(g(x_{i \in B}, t))$ ▷ evaluate network outputs for augmented inputs

$\text{loss} \leftarrow -\frac{1}{|B|} \sum_{i \in (B \cap L)} \log z_i[y_i]$ ▷ supervised loss component

$+ w(t) \frac{1}{C|B|} \sum_{i \in B} \|z_i - \tilde{z}_i\|^2$ ▷ unsupervised loss component

 update θ using, e.g., ADAM ▷ update network parameters

end for

$Z \leftarrow \alpha Z + (1 - \alpha)z$ ▷ accumulate ensemble predictions

$\tilde{z} \leftarrow Z / (1 - \alpha^t)$ ▷ construct target vectors by bias correction

end for

return θ

Tolerance to incorrect labels

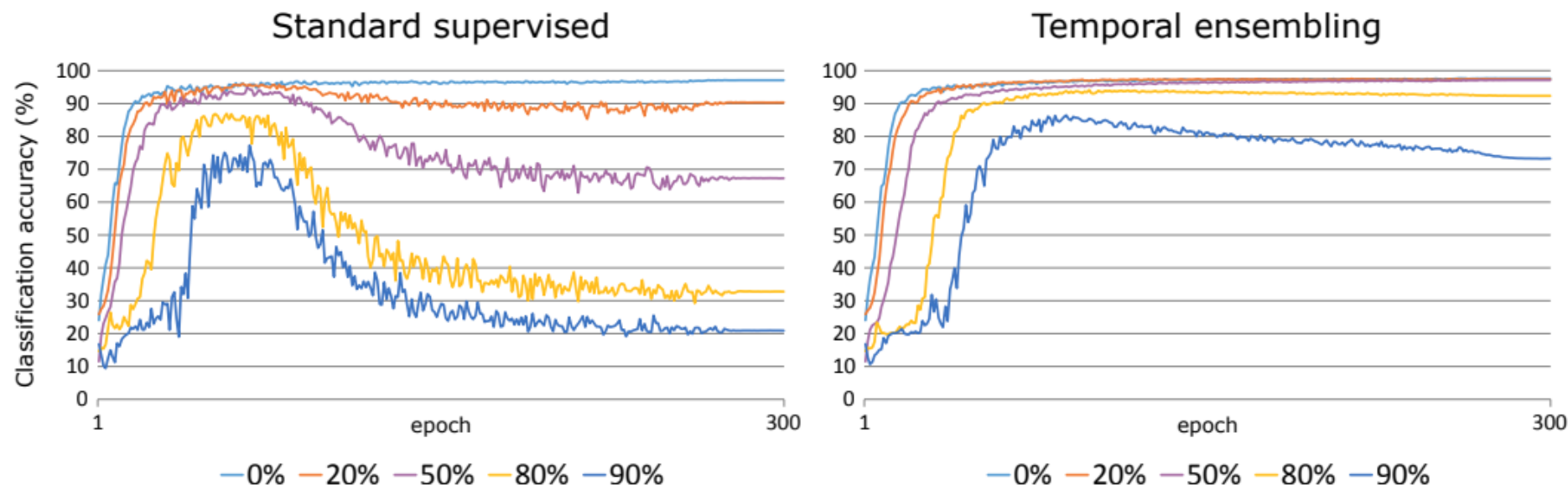


Figure 2: Percentage of correct SVHN classifications as a function of training epoch when a part of the labels is randomized. With standard supervised training (left) the classification accuracy suffers when even a small portion of the labels give disinformation, and the situation worsens quickly as the portion of randomized labels increases to 50% or more. On the other hand, temporal ensembling (right) shows almost perfect resistance to disinformation when half of the labels are random, and retains over ninety percent classification accuracy even when 80% of the labels are random.

Results

Table 1: CIFAR-10 results with 4000 labels, averages of 10 runs (4 runs for all labels).

	Error rate (%) with # labels	
	4000	All (50000)
Supervised-only	35.56 ± 1.59	7.33 ± 0.04
with augmentation	34.85 ± 1.65	6.05 ± 0.15
Conv-Large, Γ -model (Rasmus et al., 2015)	20.40 ± 0.47	
CatGAN (Springenberg, 2016)	19.58 ± 0.58	
GAN of Salimans et al. (2016)	18.63 ± 2.32	
II-model	16.55 ± 0.29	6.90 ± 0.07
II-model with augmentation	12.36 ± 0.31	5.56 ± 0.10
Temporal ensembling with augmentation	12.16 ± 0.24	5.60 ± 0.10

Table 2: SVHN results for 500 and 1000 labels, averages of 10 runs (4 runs for all labels).

Model	Error rate (%) with # labels		
	500	1000	All (73257)
Supervised-only	35.18 ± 5.61	20.47 ± 2.64	3.05 ± 0.07
with augmentation	31.59 ± 3.60	19.30 ± 3.89	2.88 ± 0.03
DGN (Kingma et al., 2014)		36.02 ± 0.10	
Virtual Adversarial (Miyato et al., 2016)		24.63	
ADGM (Maaløe et al., 2016)		22.86	
SDGM (Maaløe et al., 2016)		16.61 ± 0.24	
GAN of Salimans et al. (2016)	18.44 ± 4.8	8.11 ± 1.3	
II-model	7.05 ± 0.30	5.43 ± 0.25	2.78 ± 0.03
II-model with augmentation	6.65 ± 0.53	4.82 ± 0.17	2.54 ± 0.04
Temporal ensembling with augmentation	5.12 ± 0.13	4.42 ± 0.16	2.74 ± 0.06

Conclusions

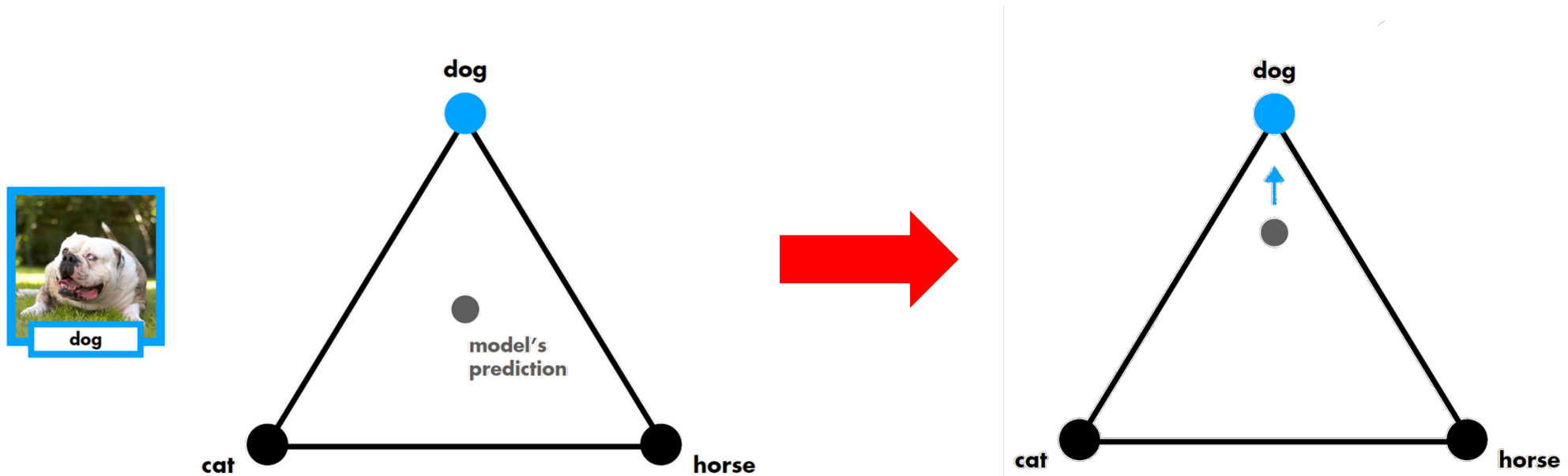
- The benefits of temporal ensembling compared to Π -model are twofold.
 - First, the training is faster because the network is evaluated only once per input on each epoch.
 - Second, the training targets $\tilde{z}$ can be expected to be less noisy than with Π -model.
- At each training step, all the **Exponential Moving Average (EMA)** predictions of the examples in that mini-batch are updated based on the new predictions. Consequently, the EMA prediction of each example is formed by an ensemble of the model's current version and those earlier versions that evaluated the same example.
- This ensembling improves the quality of the predictions, and using them as the **teacher** predictions improves results. However, since each target is updated only once per epoch, the learned information is incorporated into the training process at a slow pace.
- We indeed obtain somewhat better results with temporal ensembling than with Π -model in the same number of training epochs.
- The downside compared to Π -model is the need to store auxiliary data across epochs, and the new hyperparameter α .
- While the matrix Z can be fairly large when the dataset contains a large number of items and categories, its elements are accessed relatively infrequently. Thus, it can be stored, e.g., in a memory mapped file.

Mean Teacher

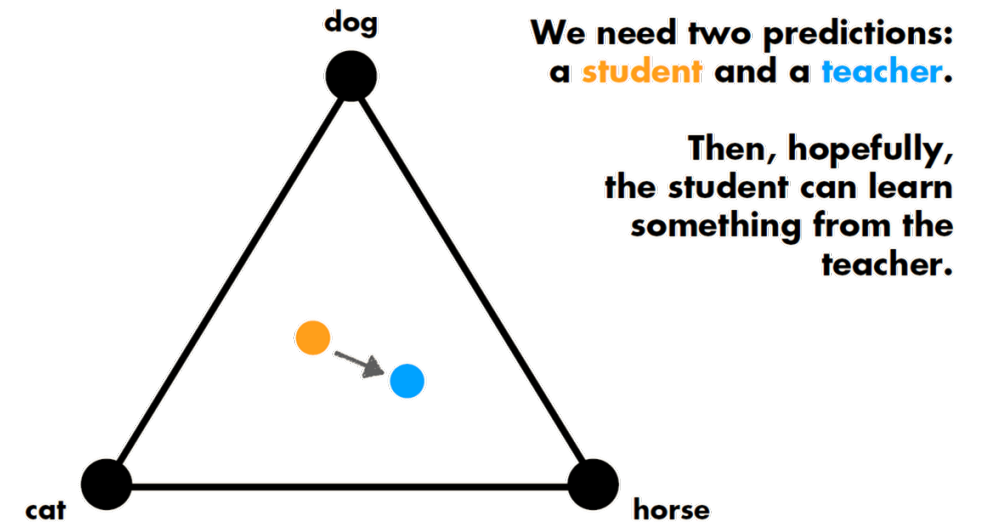
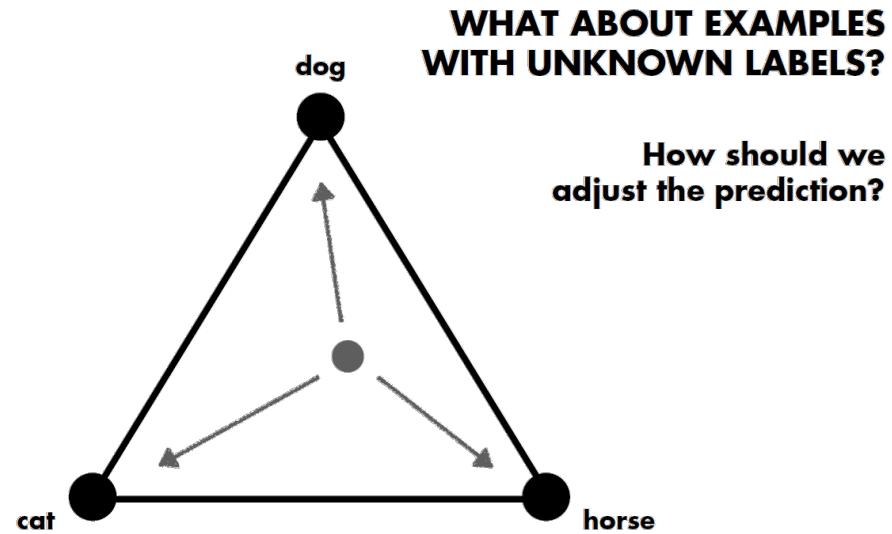
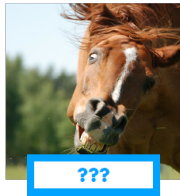
- A method that averages model weights instead of label predictions to improve semi-supervised learning. This model can be considered as a **teacher** that learns a **student** model.
- The teacher model is an average of consecutive student models, so we call it **Mean Teacher** method.
- Averaging model weights over training steps tends to produce a more accurate model than using the final weights directly
- Instead of sharing the weights with the student model, the teacher model uses the exponential moving average (**EMA**) weights of the student model: it can aggregate information after every step instead of every epoch.
- In addition, since the weight averages improve all layer outputs, not just the top output, the target model has better intermediate representations.
- As a result, Mean Teacher improves test accuracy and enables training with a fewer number of labels wrt Temporal Ensembling: without changing the network architecture, Mean Teacher achieves an error rate of 4.35% on SVHN with 250 labels, outperforming Temporal Ensembling trained with 1000 labels.

Idea

In Supervised Learning, the *True label* pulls prediction to its direction.

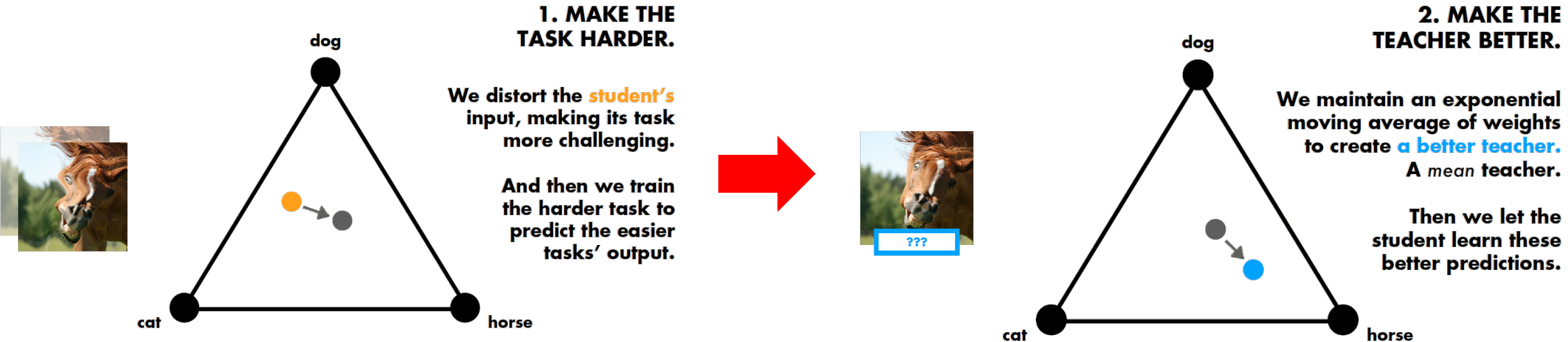


Idea



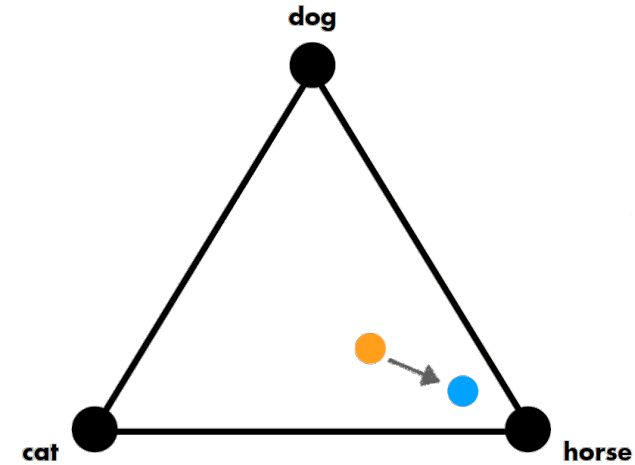
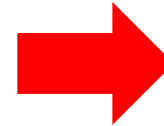
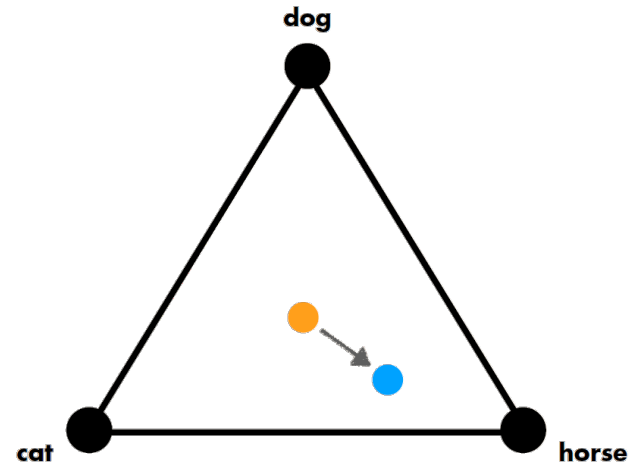
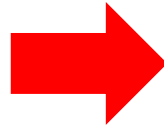
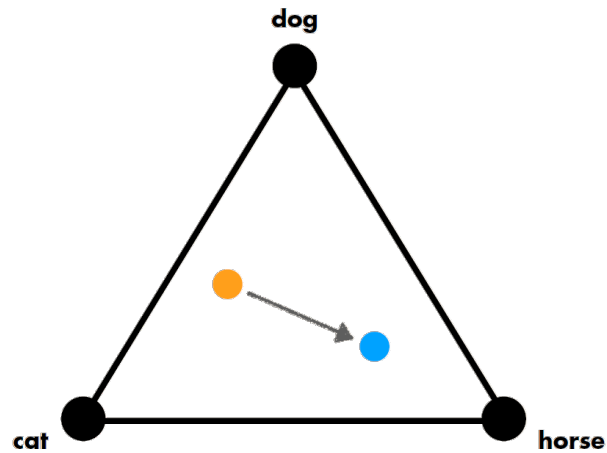
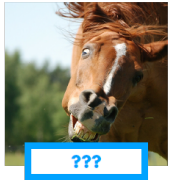
Idea

So how to do this? Two ways (non exclusive indeed) ...



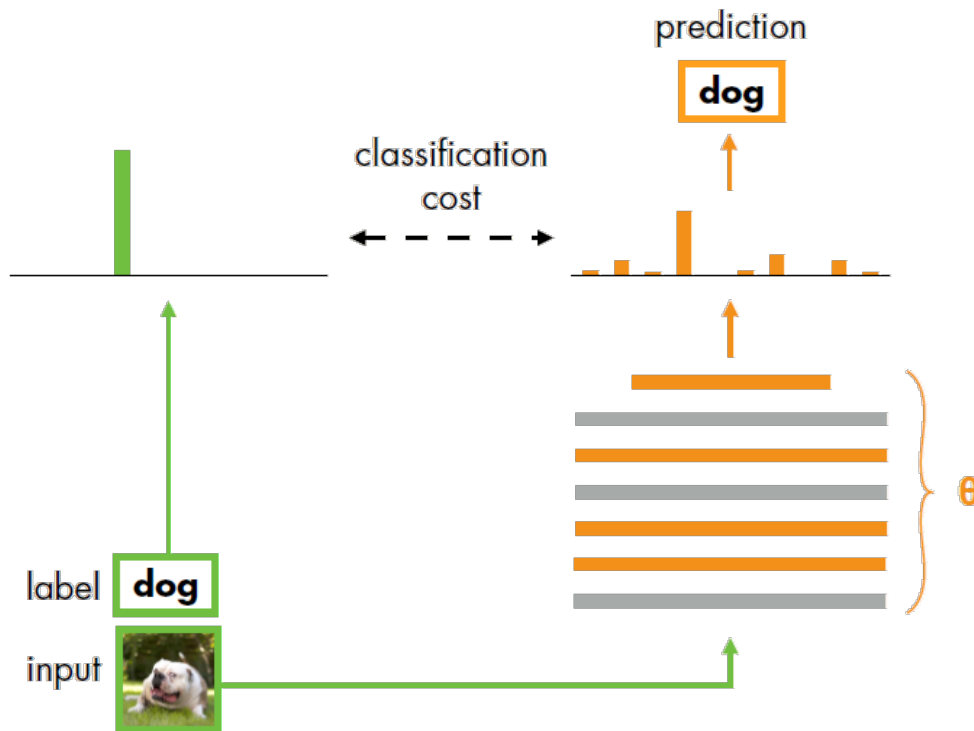
Idea

The student and the teacher improve each other in a virtuous cycle!



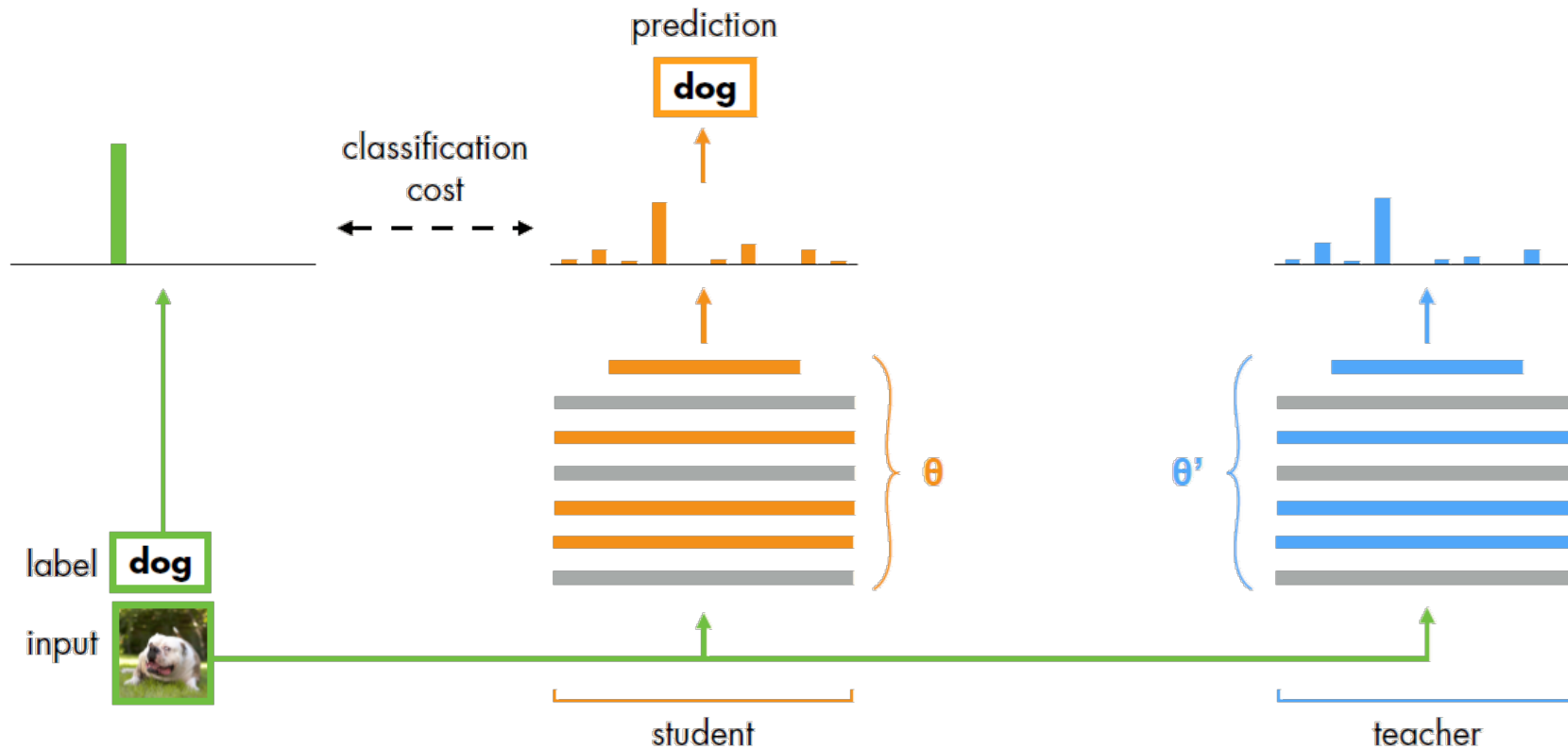
Proposed Method

Take a supervised model.



Proposed Method

Make a copy of it.

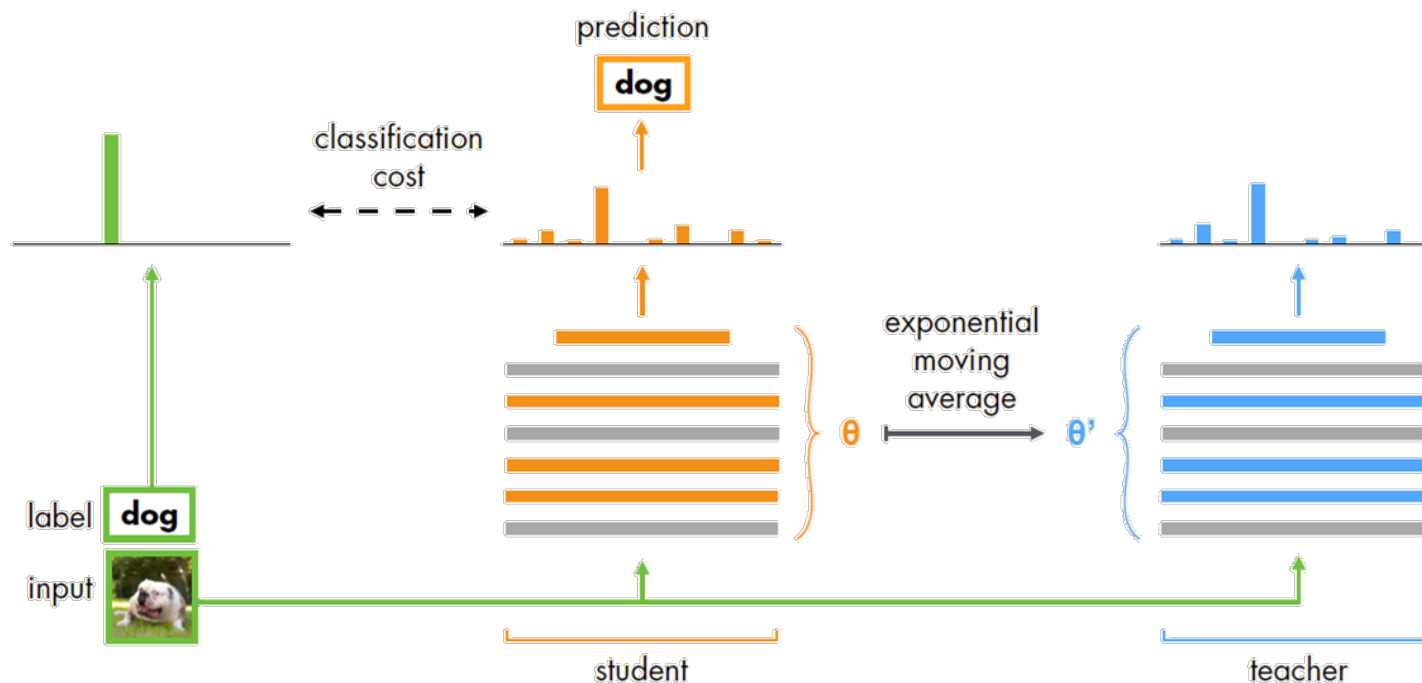


Proposed Method

With a weighted average of successive predictions, the method defines θ'_t at training step t as the EMA of successive θ weights to update teacher weights after each training step as:

$$\theta'_t = \alpha\theta'_{t-1} + (1 - \alpha)\theta_t$$

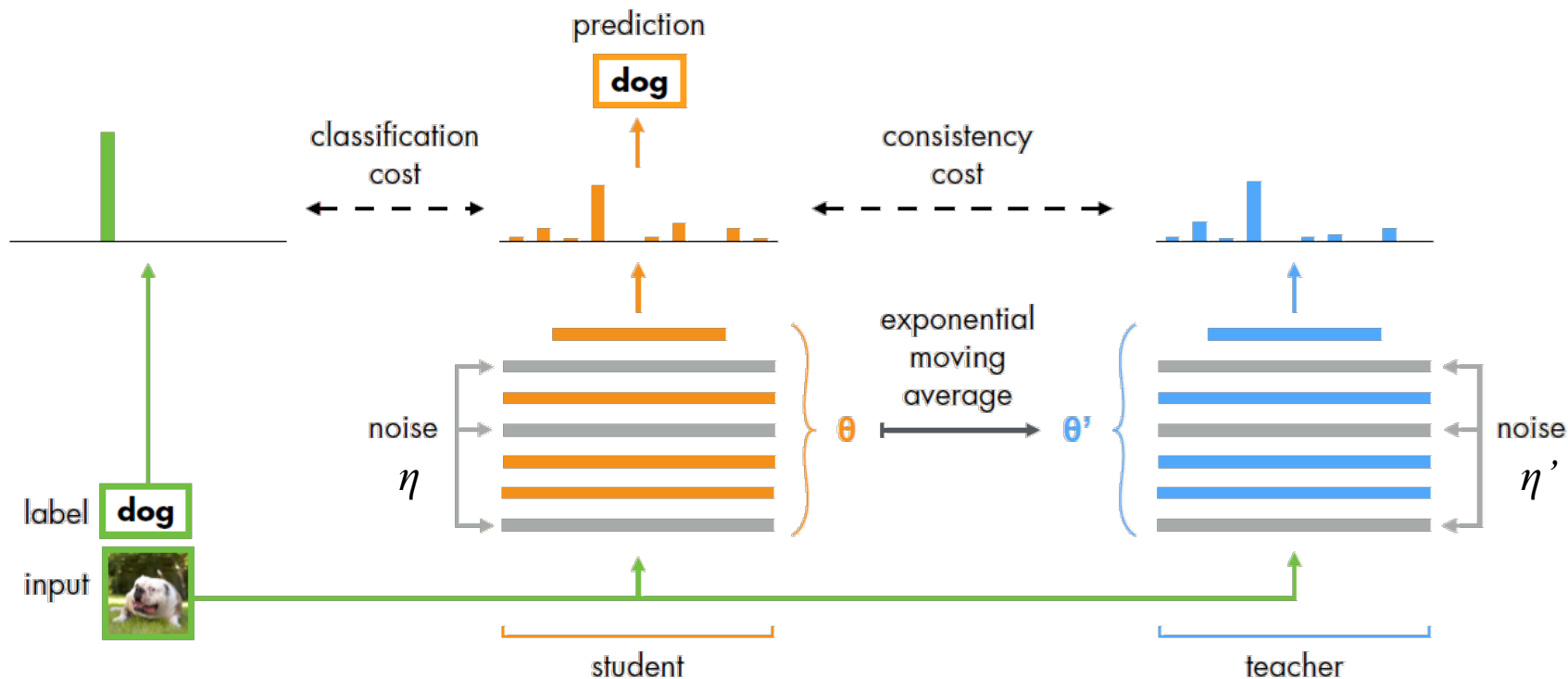
where α is a smoothing coefficient hyperparameter.



Proposed Method

The method defines *consistency cost* J as the expected distance between the prediction of the student model (with weights θ and noise η) and the prediction of the teacher model (with weights θ' and noise η').

$$J(\theta) = \mathbb{E}_{x, \eta', \eta} \left[\|f(x, \theta', \eta') - f(x, \theta, \eta)\|^2 \right]$$



Dealing with Noisy Labels

Noisy Labels



Standard supervised training

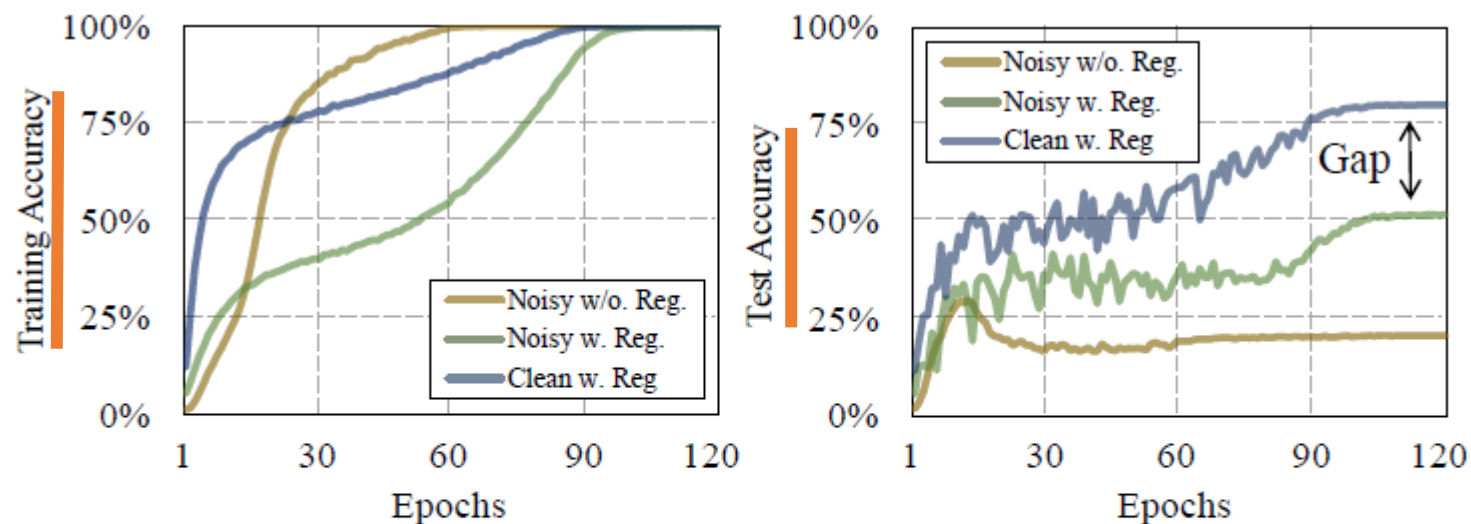


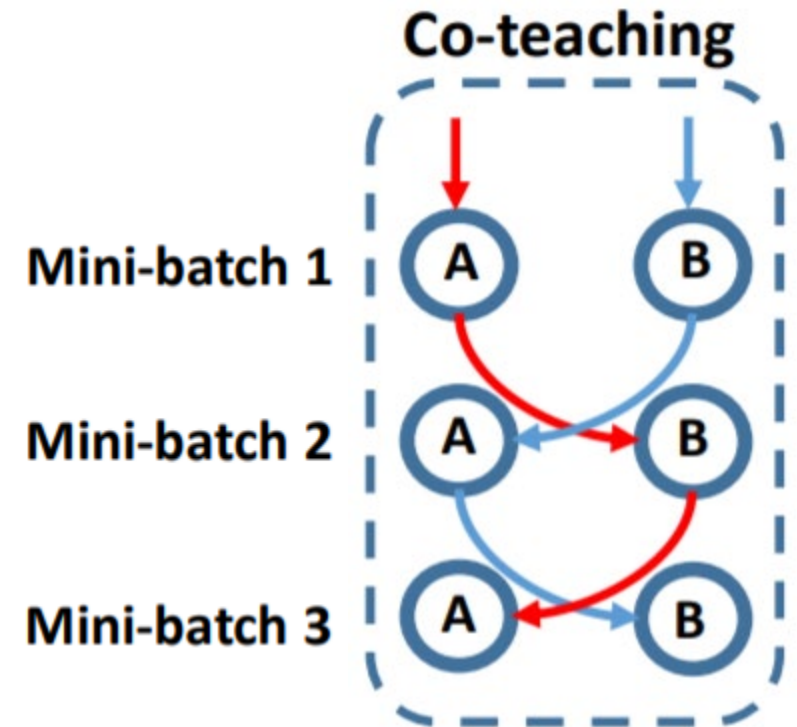
Fig. 1. Convergence curves of training and test accuracy when training WideResNet-16-8 using a standard training method on the CIFAR-100 dataset with the symmetric noise of 40%: “Noisy w/o. Reg.” and “Noisy w. Reg.” are the models trained on noisy data without and with regularization, respectively, and “Clean w. Reg.” is the model trained on clean data with regularization.

Regularization:

Data augmentation + Weight decay + Dropout + Batch normalization

Co-teaching: Robust Training of Deep Neural Networks with Extremely Noisy Labels

- The paper proposes a new deep learning paradigm called **Co-teaching** for dealing with noisy labels.
- Namely, we train two deep neural networks simultaneously, and let them teach each other given every mini-batch:
 - First, each network feeds forward all data and selects some data of possibly clean labels.
 - Second, two networks communicate with each other what data in this mini-batch should be used for training.
 - Third, each network back propagates the data selected by its peer network and updates itself.
- In Co-teaching, since two networks have different learning abilities, they can filter different types of error introduced by noisy labels.
- In this exchange procedure, the error flows can be reduced by peer networks mutually.



Co-teaching Algorithm

Co-teaching works in a mini-batch manner. Specifically, we maintain two networks f (with parameter w_f) and g (with parameter w_g). When a mini-batch $\bar{\mathcal{D}}$ is formed (step 3), we first let f (resp. g) select a small proportion of instances in this mini-batch $\bar{\mathcal{D}}_f$ (resp. $\bar{\mathcal{D}}_g$) that have small training loss (steps 4 and 5). The number of instances is controlled by $R(T)$, and f (resp. g) only selects $R(T)$ percentage of small-loss instances out of the mini-batch. Then, the selected instances are fed into its peer network as the useful knowledge for parameter updates (steps 6 and 7).

Algorithm 1 Co-teaching Algorithm.

```
1: Input  $w_f$  and  $w_g$ , learning rate  $\eta$ , fixed  $\tau$ , epoch  $T_k$  and  $T_{\max}$ , iteration  $N_{\max}$ ;  
for  $T = 1, 2, \dots, T_{\max}$  do  
    2: Shuffle training set  $\mathcal{D}$ ; //noisy dataset  
    for  $N = 1, \dots, N_{\max}$  do  
        3: Fetch mini-batch  $\bar{\mathcal{D}}$  from  $\mathcal{D}$ ;  
        4: Obtain  $\bar{\mathcal{D}}_f = \arg \min_{\mathcal{D}': |\mathcal{D}'| \geq R(T)|\bar{\mathcal{D}}|} \ell(f, \mathcal{D}')$ ; //sample  $R(T)\%$  small-loss instances  
        5: Obtain  $\bar{\mathcal{D}}_g = \arg \min_{\mathcal{D}': |\mathcal{D}'| \geq R(T)|\bar{\mathcal{D}}|} \ell(g, \mathcal{D}')$ ; //sample  $R(T)\%$  small-loss instances  
        6: Update  $w_f = w_f - \eta \nabla \ell(f, \bar{\mathcal{D}}_g)$ ; //update  $w_f$  by  $\bar{\mathcal{D}}_g$ ;  
        7: Update  $w_g = w_g - \eta \nabla \ell(g, \bar{\mathcal{D}}_f)$ ; //update  $w_g$  by  $\bar{\mathcal{D}}_f$ ;  
    end  
    8: Update  $R(T) = 1 - \min \left\{ \frac{T}{T_k} \tau, \tau \right\}$ ;  
end  
9: Output  $w_f$  and  $w_g$ .
```

Co-teaching

There are two important questions for designing the Algorithm 1:

Q1. Why can sampling small-loss instances based on dynamic $R(T)$ help us find clean instances?

Q2. Why do we need two networks and cross-update the parameters?

- To answer the first question, we first need to clarify the connection between small losses and clean instances.
- Intuitively, when labels are correct, small-loss instances are more likely to be the ones which are correctly labeled. Thus, if we train our classifier only using small-loss instances in each mini-batch data, it should be resistant to noisy labels.
- The “memorization” effect of deep networks can exactly help us address this problem. Namely, on noisy data sets, even with the existence of noisy labels, deep networks will learn clean and easy pattern in the initial epochs. So, they have the ability to filter out noisy instances using their loss values at the beginning of training.
- Yet, the problem is that when the number of epochs goes large, they will eventually overfit on noisy labels.
- To rectify this problem, we want to keep more instances in the mini-batch at the start, i.e., $R(T)$ is large. Then, we gradually increase the drop rate, i.e., $R(T)$ becomes smaller, so that we can keep clean instances and drop those noisy ones before our networks memorize them.

Co-teaching

- However, a few wrongly selected instances can deteriorate the learning performance of the whole model. This connects with our second question, where two classifiers can help.
- Intuitively, different classifiers can generate different decision boundaries and then have different abilities to learn. Thus, when training on noisy labels, we also expect that they can have different abilities to filter out the label noise.
- This motivates us to exchange the selected small-loss instances, i.e., update parameters in f (resp. g) using mini-batch instances selected from g (resp. f).
- The two networks will adaptively correct the training error by the peer network if the selected instances are not fully clean.
- Take “peer-review” as a supportive example. When students check their own exam papers, it is hard for them to find any error or bug because they have some personal bias for the answers. Luckily, they can ask peer classmates to review their papers. Then, it becomes much easier for them to find their potential faults.
- To sum up, as the error from one network will not be directly transferred back itself, we can expect that our Co-teaching method can deal with heavier noise compared with the self-evolving one.

Results

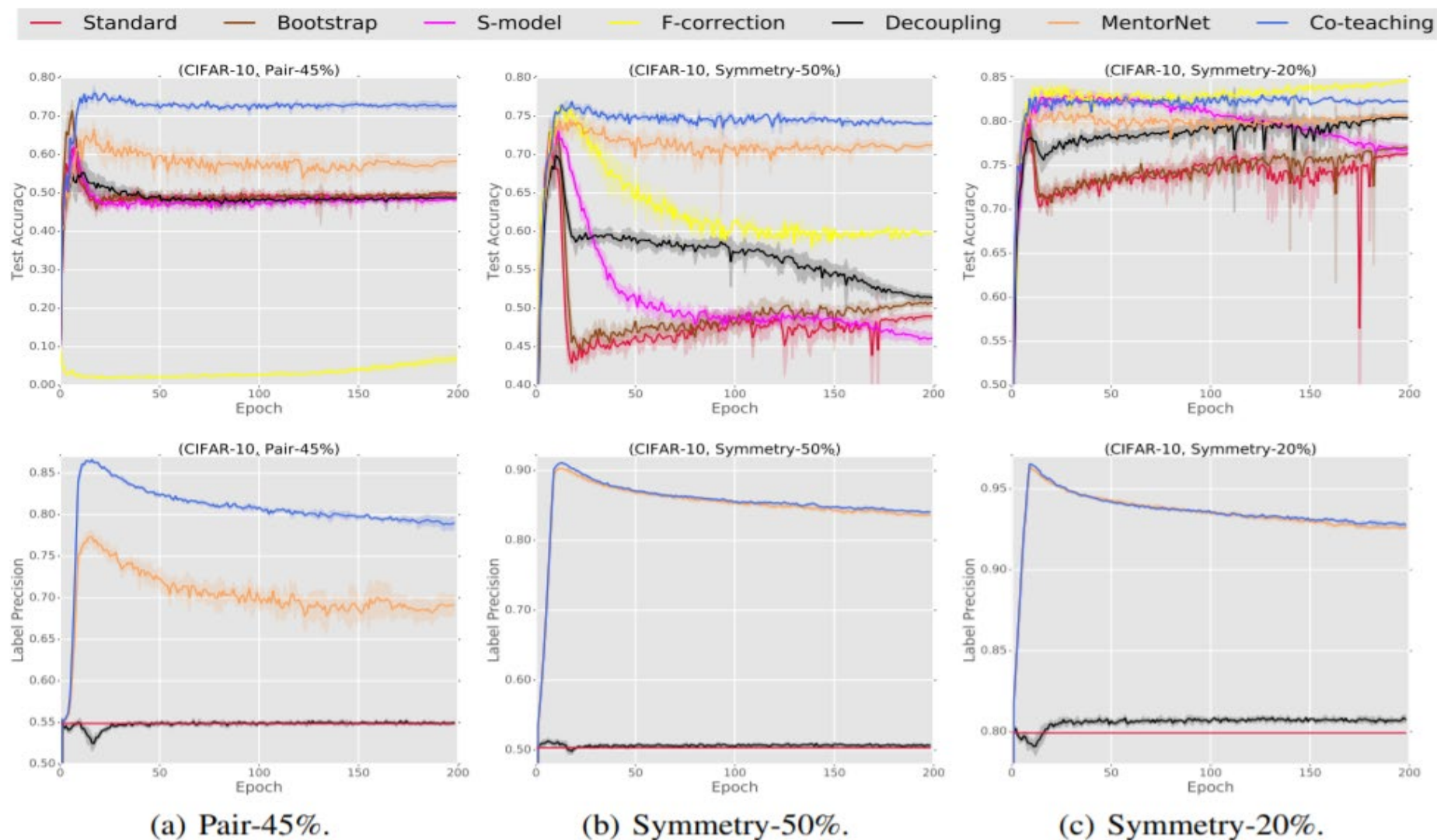


Figure 5: Results on *CIFAR-10* dataset. Top: test accuracy vs. number of epochs; bottom: label precision vs. number of epochs.